

Selection Sort

The "Search & Swap" Algorithm

DSA Mastery Series | Volume 05

codeHive

1. Core Concept

Selection Sort is a straightforward algorithm that organizes data by repeatedly finding the **minimum value** from the unsorted part of the list and moving it to the front.

Unlike Bubble Sort, which compares adjacent pairs, Selection Sort scans the entire unsorted section to find the single smallest element before making a move.

How it Works

1. Find the lowest value in the array.
2. Swap it with the first value of the unsorted part.
3. Repeat for the remaining unsorted values until only one element is left.

2. Manual Trace

Let's trace the array: [7 , 12 , 9 , 11 , 3]

Step 1: Initial search. The lowest value is 3.

Step 2: Move 3 to the front (swap with 7).

[3 , 12 , 9 , 11 , 7]

Step 3: Search remaining [12, 9, 11, 7]. Lowest is 7.

Step 4: Swap 7 with 12.

```
[ 3, 7, 9, 11, 12 ]
```

Step 5: Search remaining [9, 11, 12]. Lowest is 9. It is already in place.

Step 6: Search remaining [11, 12]. Lowest is 11. Already in place.



Key Observation

With an array of size **N**, the algorithm needs to perform **N-1** passes to guarantee the list is fully sorted. The unsorted portion shrinks by exactly one element after every pass.

3. The Efficiency Trap

There are two ways to move the minimum value to the front. One is intuitive but **slow**, the other is **efficient**.

Method A: Shifting (The Problem)

If you remove the lowest value (using `pop ( )`) and insert it at the front (using `insert ( )`), the computer must shift all intermediate values to fill the gap and make space.

This background work causes significant slowdowns in large datasets.

Method B: Swapping (The Solution)

Instead of shifting everyone, simply **swap** the first element of the unsorted part with the minimum element found. It doesn't matter where the "non-minimum" value lands, as it is still in the unsorted portion.

4. Python Implementation

This implementation uses the optimized **Swap** approach, which is the standard for Selection Sort.

OPTIMIZED SWAP VERSION

```
my_array = [64, 34, 25, 12, 22, 11, 90, 5]
n = len(my_array)

for i in range(n):
    # Assume the first unsorted element is the minimum
    min_index = i

    # Iterate through the rest to find the actual minimum
    for j in range(i+1, n):
        if my_array[j] < my_array[min_index]:
            min_index = j

    # Swap the found minimum with the first unsorted element
    my_array[i], my_array[min_index] = my_array[min_index], my_array[i]

print("Sorted array:", my_array)
```

5. Time Complexity Analysis

Selection Sort performs two nested loops. The outer loop runs n times, and the inner loop runs roughly $n/2$ times on average.

Total Complexity: **$O(n^2)$**

Selection Sort vs. Bubble Sort

Feature	Selection Sort	Bubble Sort
Worst Case	$O(n^2)$	$O(n^2)$
Best Case	$O(n^2)$	$O(n)$
Swaps	Fewer (Max $n-1$)	Many (Up to n^2)

Selection Sort is unique because even if the array is already sorted, it will still scan every element to confirm it has found the minimum.

6. Exercise

Question: Using Selection Sort on the array `[ 7, 12, 9, 11, 3 ]`, what is the value of the **LAST** element in the array after the **FIRST** complete pass?

Answer: 7 (3 and 7 were swapped)

MASTERED

You have successfully completed the Selection Sort Module.